CE152: Object-Oriented Programming

Reading files and exceptions

Dr. Sebastian Halder

CSEE

2023

Input and output

Devices

- Programs can interact with a variety of devices to read or write Data
- Input devices:
  - keyboard, mouse, terminal, GUI, touch screen, hard disk, network
- Output devices:
  - terminal, GUI, hard disk, network, printer, sound

Reading text files

- Text files are made up from characters encoded according to a particular standard (e.g. UTF-8)
- Text files have one or more lines
- Can be edited using a text editor
- Example: Java source files, CSV files...

Reading binary files

- Will contain a sequence of bytes
- If you open them in a text editor we will see strange symbols and many “blocks”
- This means we cannot easily edit them
- Must know the encoding format to use
- Example: Java byte code files
- We will focus on text files

Data streams

- In Java if we want to read from something we have to open a *stream*
  - These can be *input or output streams*
- Streams can be connected to all kinds of sources such as files, devices or the network
- The data can be *low level (bytes)* or *high level (characters)*
- Streams may be buffered to improve performance
  - That means several items are transferred together instead of one at a time
  - Transferring a set of items is called *flushing*

Standard streams

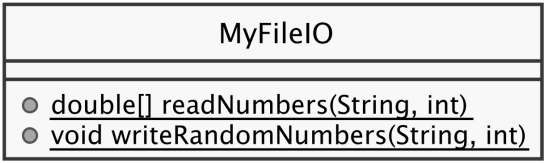
- *Standard streams* are usually keyboard input and output to the console
- We have used two streams already:
  - Standard output: *System.out* which is a `PrintStream`
  - Standard input: *System.in* which is an `InputStream`
- Java has one additional standard stream that we have not used:
  - Standard error: *System.err* which is also a `PrintStream`
- `PrintStream` writes text
- `InputStream` reads bytes
  - We have been wrapping it in a Scanner object to decode them

Recommended approach to Text I/O

- Read data from a text file
  - Create a `File` Object and wrap it in a Scanner
  - Use the `Scanner` methods (`next`, `nextInt`,...) to read data
  - Close the scanner when no longer required
- Write data to a text file
  - Create a `PrintStream` Object to the file
  - Use `PrintStream` methods (`println`, `print`, `printf`,...) to write data
  - Close the stream when not longer required
- When interacting with files `Exceptions` are very helpful which we will learn about later

Our I/O class

- Let's make a class called MyFileIO that can:
  - write random numbers to a file
  - read the numbers from the file



Importing the required packages

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;
import java.util.Arrays;
import java.util.Scanner;
```

Method headers

```
public class MyFileIO {
    public static double[] readNumbers(
            String fileName, int n)
            throws FileNotFoundException {
              ...
    }

    public static void writeRandomDoubles(
            String fileName, int n)
            throws FileNotFoundException {
              ...
    }
}
```

Reading the file

```java
public static double[] readNumbers(
      String fileName, int n)
      throws FileNotFoundException {
    Scanner input = new Scanner(new File(fileName));
    double[] tmp = new double[n];
    int i = 0;
    // Could this be a for loop?
    while (input.hasNextDouble() && i < n) {
        tmp[i++] = input.nextDouble();
    }
    input.close();
    // Why are we using i and not n?
    return Arrays.copyOf(tmp, i);
}
```

Writing the file

```
public static void writeRandomDoubles(
        String fileName, int n)
        throws FileNotFoundException {
    PrintStream output = new PrintStream(fileName);
    for (int i = 0; i < n; i++) {
        output.println(Math.random());}
    output.close();
}
```

Using the methods

```java
import java.io.FileNotFoundException;
public class Main {
// Main also has to throw the exception or the compiler
// will complain
    public static void main(String[] args)
            throws FileNotFoundException {

        String fileName = "numbers.txt";
        MyFileIO.writeRandomDoubles(fileName, 100);
        double[] numbers
          = MyFileIO.readNumbers(fileName, 100);
        for (double d : numbers)
            System.out.println(d);
    }
}
```

Remarks

- If the file (in this case numbers.txt) does not exist *PrintStream* will create it
- Existing content will be overwritten
- If we want to append to an existing file we need to wrap *PrintStream* around a suitable *FileOutputStream* with a boolean specifying append true or false

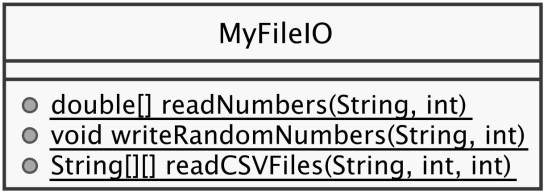
```
PrintStream output = new PrintStream(
    new FileOutputStream(fileName, true));
```

- *PrintStream* uses an output buffer that will be flushed when full
- *PrintStream* Objects can be created so that they flush automatically after each line
- For further details on this such as character encoding please see literature

Reading CSV files

- Often data is stored in *Comma Separated Values* or *CSV files*
- This is a simple text file format to hold tabular data
- Each line contains one row of the table
- The columns of the table are separated by commas
- Other delimiters such as semicolons may be used (e.g. if we want to use the comma for something else)
- Quotations may be used to separate numerical from text fields

Adding reading CSV files to MyFileIO



Read CSV in Java

```
public static String[][] readCSVFile(String fileName,
                int rows, int cols, String delimiter)
                throws FileNotFoundException{
    Scanner input = new Scanner(new File(fileName));
    String[][] tmp = new String[rows][cols];
    String curLine;
    int i = 0;
    while (input.hasNextLine() && i < rows) {
        curLine = input.nextLine();
        tmp[i++] = Arrays.copyOf(curLine.split(delimiter),
        cols);
    }
    input.close();
    return Arrays.copyOf(tmp, i);
}
```

Exceptions

What are exceptions?

- An *exception* is an interruption of the normal flow of the program
  - For example if the file we want to read does not exist
- Methods (such as our FileIO methods) can *throw exceptions*
- After an exception is thrown it must be *handled*
  - This can be via a *try-catch* block which will allow the program to continue running
  - If there is no *exception handler* the program will terminate and output information on the exception (a so-called *stack trace*)
- As we saw in the previous examples if a method throws an exception it must be passed all the way to the main method (or handled) otherwise the code will not compile

Exception handling

- There are three possible blocks of code involved in exception handling
  - *try*: this is the initial code in which the exception may occur
  - *catch*: this is executed if the initial code throws an exception (there can be multiple catch blocks!)
  - *finally*: this is always executed regardless of whether an exception occurs
- Since Java 7 "try-with-resources" was introduced which is designed for closing streams, have a look at the Java tutorial if interested!

Java code structure

```
try{
  // code that may throw exception
}
catch (ExceptionType name){
  // alternative 1
}
...
catch (ExceptionType name){
  // alternative n
}
finally {
  // cleaning up (e.g. closing stream)
}
```

Adding exception handling to writeRandomDoubles

```
public static void writeRandomDoubles(
        String fileName, int n) {
    PrintStream output = null;
    try {
        output = new PrintStream(fileName);
        for (int i = 0; i < n; i++) {
            output.println(Math.random());}
    }
    catch (FileNotFoundException e) {
        System.out.println("File not found: "
        + e.getMessage());
    }
    finally {
        output.close();}}
```

We can also handle the exception in main()

```
try {
    double[] numbers = MyFileIO.readNumbers(fileName, 100);
    for (double d : numbers)
        System.out.println(d);
}
catch (FileNotFoundException e){
    System.out.println("File not found: " + fileName);
}
```

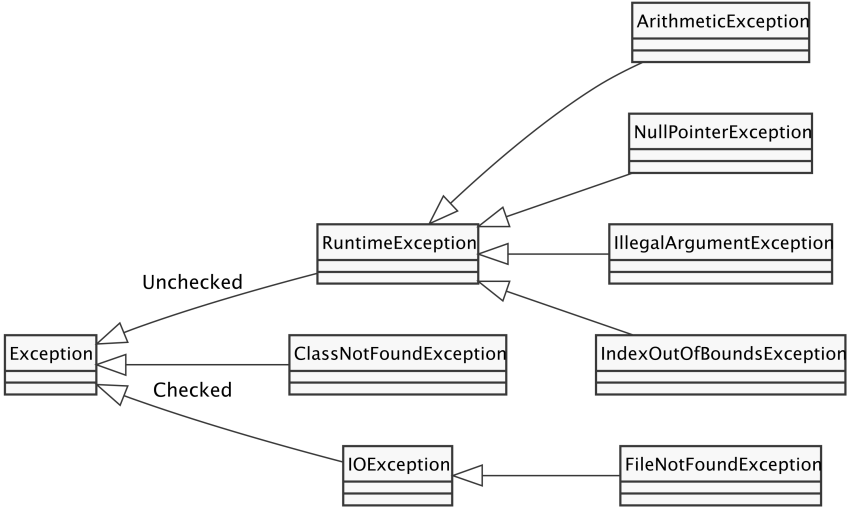
Checked vs. unchecked Exceptions

- Java has *checked* and *unchecked* exceptions
- *Checked exceptions* will cause compiler errors unless:
  - *handled* in the method (using *try-catch*)
  - *declared* in the method header (using *throws exception*)
- *Unchecked exceptions* do not cause compiler errors
- Do not need to be declared in method header
  - Advantage: shorter code
  - Disadvantage: possible to overlook
- By the way, *checked exceptions* are a controversial feature because:
  - *Scalability*: the number of exceptions we may need to deal with can become quite large
  - *Versioning*: adding a new exception may force me to change the code everywhere this exception may be thrown or caught

Exception types and messages

- Exception is a class with several subclasses
- Checked exceptions include IOException, FileNotFoundException and ClassNotFoundException
- Unchecked exceptions include RuntimeException and IndexOutOfBoundsException (when does the latter occur?)
- There is also a class called *Error*
  - Can be thrown
  - Intended for serious problems that cannot be caught
  - Please see documentation if interested
- Of course we can create our own exception class!

Exception UML



Throwing exceptions

- Sometimes we want to throw our own exceptions and not just react to existing exceptions
- For this we can use the keyword *throw*

```
public static void testThrowException(int i) {
  if (i < 0)
    throw new IllegalArgumentException("i=" + i);
  else
    System.out.println("All is fine.");
}
```

Including a message

- We can create a standard exception and include a message

```
throw new Exception("Something's wrong!");
```

- We can retrieve this exception when we handle the exception

```
catch (Exception e){
  System.out.println(e.getMessage());
}
```

Where to handle exceptions

- The exceptions should not be handled where they are thrown (in most cases)
- The best place to handle the exception
  - Remedial action can be taken
  - The current interaction or transaction can be cleanly terminated
- For example in an interactive program the user interface is a good place
  - The user can be alerted
  - Alternatives can be presented

Live coding

- Implement file read and write
- Exception handling vs. stack trace
- Using exceptions to prompt the user to re-enter the filename

Summary

Summary

- Reading and writing files is critical for many applications
- Exceptions are used not only in this scenario but in many others as well
- Please visit the support class and ask me any questions you may have

Thank you and see you in the next video!